I R I S

Intelligent Real-time Interactive System

A voice-first AI assistant that acts as your hands — listening, thinking, and executing tasks on your PC.

AUTHORS
Aradhya · Aryan · Maneesh

LICENSE
MIT

STATUS
Pre-Launch

PLATFORM
Windows 10/11 + macOS

v0.2.1 · May 2026

Table of Contents

I R I S

Intelligent Real-time Interactive System

A voice-first AI assistant that acts as your hands — listening, thinking, and executing tasks on your PC.

AUTHORS
Aradhya · Aryan · Maneesh

LICENSE
MIT

STATUS
Pre-Launch

PLATFORM
Windows 10/11 + macOS

v0.2.1 · May 2026

1. Project Overview

IRIS (**Intelligent Real-time Interactive System**) is an open-source, voice-controlled AI assistant for Windows and macOS. It bridges natural language commands and system actions through a modular, async-first architecture.

What IRIS Does

- **Voice Input** — Speak naturally; IRIS listens via always-on mic with wake word detection
- **AI Reasoning** — DeepSeek V3 Flash/Pro powered planning and task decomposition
- **System Actions** — Execute files, apps, browser automation, shell commands
- **Memory** — Learns from past interactions via ChromaDB vector store + knowledge graph
- **Real-time UI** — Transparent Tauri overlay with animated state feedback border

Current Status

Phase	Scope	Status
Phase 1	Core audio, LLM, agents, 12 action handlers, memory, UI overlay	<font color="#22C55E"><b>COMPL
Phase 2	Email, todo, weather, timer, self-improving loop	<font color="#F59E0B"><b>IN PRO
Phase 3	Performance optimization, E2E testing, security hardening	<font color="#64748B"><b>PLANNE

2. Vision & Goals

Short-term (3-6 months)

- Stable, production-ready voice assistant on Windows + macOS
- 25+ reliable actions (files, apps, browser, email, scheduling)
- Sub-3-second end-to-end latency (speak to response)
- Comprehensive documentation and contributor guide

Medium-term (6-12 months)

- Calendar + smart home integration
- Linux full support
- Advanced action chaining (multi-step workflows)
- Settings UI (no config file editing)
- Community extensions / plugin system

Long-term (1-2 years)

- Mobile companion app (iOS/Android)
- Multi-user / enterprise features
- Vision capabilities (screen understanding via LLM vision)
- Open ecosystem with third-party action marketplace
- Multi-language support

Ultimate Vision

"A PC assistant that understands you, remembers you, and acts for you — as natural as talking to a coworker."

3. Core Architecture

High-Level Pipeline

The full pipeline from voice input to spoken output:

#	Module	Implementation
1	Audio Capture	MicListener (sounddevice, 16kHz mono float32)
2	Wake Word	ASR-based text matching for 'Iris' / 'Jarvis'
3	Speech-to-Text	Groq Whisper API (whisper-large-v3-turbo)
4	State Manager	IDLE -> INTERACTIVE -> ACTING -> IDLE
5	Memory Injection	ChromaDB semantic search + short-term buffer
6	LLM Router	DeepSeek Flash (planning) / Pro (complex reasoning)
7	Agent Planner	Decomposes goal into subtask JSON
8	Agent Executor	Runs subtasks: voice, browser, code, actions
9	Action Router	Safety check -> approval gate -> execute handler
10	TTS Output	ElevenLabs streaming with interrupt support
11	UI Update	WebSocket IPC -> Tauri overlay border animation

Key Design Principles

Principle	Detail
Async-First	All I/O is non-blocking — audio, API calls, file ops use asyncio
Safety by Default	Dangerous actions require explicit user approval via overlay popup
Graceful Degradation	Modules load with try/except stubs; partial boot always works
Memory-Aware	Past interactions injected into every LLM prompt via semantic search
Modular	New actions = 1 file + 2 line registration (handler + safety map)
Cross-Platform	pathlib, platform guards, macOS/Windows-specific code isolated
Privacy-First	No data leaves your PC except to APIs you explicitly configure
Self-Improving	IRIS will inspect conversation logs and refine its own prompts (Phase 2)

Module Organization (Actual Repo)

```
iris/
+-- audio/ # Audio input & wake word
| +-- listener.py # Mic stream (sounddevice)
| +-- asr.py # ASR engine interface
| +-- groq_whisper_backend.py # Groq cloud Whisper
| +-- wake_word.py # Wake word detection
| +-- interrupt_handler.py
+-- core/ # Event loop & state
| +-- event_loop.py # Main async loop
| +-- state_manager.py # IDLE/INTERACTIVE/ACTING/STOPPING
| +-- task_orchestrator.py
| +-- daemon.py # Signal handlers
+-- llm/ # Language models
| +-- router.py # DeepSeek-only routing
| +-- prompt_manager.py
| +-- providers/
| +-- deepseek_provider.py
+-- agents/ # AI agent system
| +-- planner.py # Task decomposition
| +-- executor.py # Subtask execution
| +-- agent_manager.py # Coordination
| +-- coding_agent.py # Code gen & run
+-- actions/ # 12 action handlers
| +-- action_router.py # Central dispatch + approval gate
| +-- safety.py # SAFE / WARN / DANGEROUS
| +-- file_actions.py # read/write/move/delete
| +-- os_actions.py # open_app, focus_window
| +-- shell_actions.py # Command execution
| +-- clipboard_actions.py
| +-- screen_actions.py # Screenshot + OCR
+-- memory/ # Persistent memory
| +-- memory_manager.py # Central interface
| +-- short_term.py # Session buffer (20 msgs)
| +-- long_term.py # ChromaDB vectors
| +-- vector_store.py # BGE-M3 embeddings
| +-- graph.py # NetworkX knowledge graph
+-- browser/ # Browser automation
| +-- browser_agent.py # Playwright wrapper
| +-- login_handler.py # Encrypted credentials
| +-- scraper.py
+-- voice/ # Text-to-speech
| +-- tts_router.py
| +-- elevenlabs_tts.py # Streaming + interruptible
| +-- stream_player.py
+-- ui/ # Frontend
| +-- ipc_bridge.py # WebSocket bridge (port 7788)
| +-- tauri-app/ # Tauri v2 overlay
+-- utils/ # Shared utilities
| +-- config.py, logger.py, platform.py
+-- configs/
| +-- settings.yaml
| +-- keys.env # API keys (gitignored)
+-- main.py # Entrypoint
```

4. Implemented Features

Audio & Voice

Feature	Implementation	Status
Wake word detection	ASR text matching ('Iris' / 'Jarvis')	<font color="#22C55E"><b>Done<
Microphone input	sounddevice (16kHz, float32, mono)	<font color="#22C55E"><b>Done<
Speech-to-text	Groq Whisper API (whisper-large-v3-turbo)	<font color="#22C55E"><b>Done<
Interrupt handling	Keyword 'stop' / 'cancel' mid-speech	<font color="#22C55E"><b>Done<
Text-to-speech	ElevenLabs streaming (eleven_turbo_v2)	<font color="#22C55E"><b>Done<
Energy-gated ASR	RMS silence detection skips silent buffers	<font color="#22C55E"><b>Done<

AI & Reasoning

Feature	Implementation	Status
LLM routing	DeepSeek Flash (fast) + Pro (complex)	<font color="#22C55E"><b>Done<
Task planning	Planner agent — JSON subtask decomposition	<font color="#22C55E"><b>Done<
Task execution	Executor agent — voice/browser/code/action dispatch	<font color="#22C55E"><b>Done<
Code generation	Coding agent — write, run, iterate	<font color="#22C55E"><b>Done<
Context injection	Memory retrieval into every prompt	<font color="#22C55E"><b>Done<

Action Handlers (12 registered)

Action	Handler	Safety
open_app	os_actions — launch applications	SAFE
focus_window	os_actions — bring window to front	SAFE
read_file	file_actions — read text files	SAFE
write_file	file_actions — create/overwrite	WARN
move_file	file_actions — move/rename	WARN
delete_file	file_actions — delete (approval required)	DANGEROUS
run_shell	shell_actions — execute commands	WARN
run_shell_sudo	shell_actions — admin commands	DANGEROUS
screenshot	screen_actions — capture screen (mss)	SAFE
ocr	screen_actions — extract text (easyocr)	SAFE
get_clipboard	clipboard_actions — read clipboard	SAFE
set_clipboard	clipboard_actions — write clipboard	WARN

Memory System

- **Short-term** — Last 20 messages in session buffer, injected into every LLM call
- **Long-term** — All conversations vectorized with BGE-M3, stored in ChromaDB
- **Knowledge Graph** — Facts and relationships extracted via NetworkX

Browser Automation

- Playwright-based navigation, clicking, typing, form filling
- Encrypted credential storage (~/.iris/credentials.enc)
- Web scraping and data extraction
- JavaScript execution in page context

UI Overlay

IRIS State	Border Color	Animation
IDLE	White	Static — waiting for wake word
INTERACTIVE	Blue #3B82F6	Slow pulse — listening to you
ACTING	Green #22C55E	Sweeping — processing task
STOPPING	Fading white	Fade out — shutting down

5. Planned Features (Phase 2)

These features are designed but **not yet implemented**. They are the immediate next priority.

Feature	Description	Owner	Est.
Email Actions	send_email, check_email — Gmail/Outlook SMTP/IMAP	Maneesh	3 days
Todo Manager	add_task, list_tasks, mark_complete, delete — persistent JSON	Maneesh	2 days
Weather API	get_weather, get_forecast — OpenWeatherMap free tier	Maneesh	1 day
Timer / Reminders	set_timer, set_reminder, list, cancel — async countdown	Maneesh	2 days
Self-Improving Loop	IRIS analyzes its own logs and refines prompts/behavior	Aradhya	5 days
DeepSeek API Key	Wire production key, test all task_types with real API	Aryan	1 day
Action Chaining	Pipe output of one action into the next	Aryan	3 days

6. System Requirements

Hardware

Component	Minimum	Recommended
CPU	Intel i5 / AMD Ryzen 5 (4 cores)	i7 / Ryzen 7
RAM	8 GB	16 GB
Storage	10 GB SSD	20 GB SSD
Microphone	Any built-in or USB mic	Dedicated headset
Internet	Required for all APIs	Stable broadband

Software

Requirement	Version	Purpose
Python	3.11+	Runtime
Windows or macOS	Win 10/11 or macOS 12+	OS
Rust + Cargo	Latest stable	Tauri build
Git	Any	Version control
FFmpeg	Any	Audio processing

API Keys

API	Purpose	Cost	Required?
DeepSeek	LLM reasoning (Flash + Pro)	~$0.01/1M tokens	YES
ElevenLabs	Voice synthesis (TTS)	Free tier: 10k chars/mo	YES
Groq	Speech-to-text (Whisper)	Free tier: 3600 RPM	YES
OpenWeatherMap	Weather data (Phase 2)	Free tier: 1000/day	Phase 2

7. Installation

Step 1 — Clone

```
git clone https://github.com/Aradhya648/Iris.git
cd Iris
```

Step 2 — Install Python dependencies

```
pip install -r requirements.txt
python -m playwright install chromium
```

Step 3 — Install Tauri CLI

```
cargo install tauri-cli # ~8 minutes first time
```

Step 4 — Create API keys file

Create **configs/keys.env** (gitignored — never committed):

```
DEEPSEEK_API_KEY=your_deepseek_key_here
ELEVENLABS_API_KEY=your_elevenlabs_key_here
GROQ_API_KEY=your_groq_key_here
```

Step 5 — Verify

```
python -c "import loguru, httpx, chromadb, websockets, sounddevice; print('All deps OK')"
python main.py # should print 'IRIS ready'
```

8. Configuration

Main config: **configs/settings.yaml**

```
llm:
default_model: "deepseek-flash"
task_routing:
plan: "deepseek-flash"
chat: "deepseek-flash"
code: "deepseek-pro"
reason: "deepseek-pro"

asr:
model: "whisper-large-v3-turbo"

voice:
elevenlabs_voice_id: "21m00Tcm4TlvDq8ikWAM"
elevenlabs_model: "eleven_turbo_v2"
stream: true

memory:
chroma_path: "~/.iris/memory/chroma"
embedding_model: "BAAI/bge-m3"
short_term_limit: 20

ui:
ipc_port: 7788

logging:
level: "DEBUG"
log_file: "logs/iris.log"
```

9. Usage Guide

Starting IRIS

Terminal 1 — Python backend:

```
cd Iris
python main.py
```

Terminal 2 — Tauri overlay:

```
cd Iris/ui/tauri-app
cargo tauri dev
```

Expected boot output:

```
IRIS booting...
ASR: Groq Whisper cloud loaded
IRIS ready. Say 'Jarvis' or 'Iris' to begin.
IPC bridge starting on ws://localhost:7788
IRIS event loop running
```

Voice Commands

Say **"Iris"** to wake, then speak your command:

Command	What Happens
"What time is it?"	DeepSeek plans -> voice action -> ElevenLabs speaks answer
"Open Notepad"	OS action -> launches app via shell
"Take a screenshot"	Screen action -> saves screenshots/screen.png
"What's in my clipboard?"	Clipboard action -> speaks contents
"Delete file test.txt"	Approval popup -> DANGEROUS -> user confirms or denies
"Stop" (while speaking)	TTS cuts off, border fades, returns to IDLE

10. API Reference

All actions follow this interface:

```
result = await action_router.execute(subtask)
# Returns: {"status": "ok"|"error", "result": str, "requires_approval": bool}
```

Action Examples

```
# Read file
await action_router.execute({
"action_type": "read_file",
"params": {"file_path": "report.txt"}
})

# Open app
await action_router.execute({
"action_type": "open_app",
"params": {"app_name": "Notepad"}
})

# Delete file (DANGEROUS — triggers approval popup)
await action_router.execute({
"action_type": "delete_file",
"params": {"file_path": "temp.txt"}
})
```

Safety Levels

Level	Behavior	Examples
SAFE	Auto-execute, no prompt	read_file, open_app, screenshot, get_clipboard
WARN	Execute + log warning	write_file, run_shell, set_clipboard, browser_click
DANGEROUS	Approval popup required	delete_file, run_shell_sudo, browser_login, run_code

11. Development Guide

Adding a New Action (4 steps)

Step 1: Create action module

```
# actions/my_action.py
async def my_action(**params) -> dict:
result = do_something(params['input'])
return {"status": "ok", "result": result}
```

Step 2: Register in action_router.py

```
ACTION_HANDLERS = {
...
"my_action": my_action.my_action,
}
```

Step 3: Classify safety in safety.py

```
ACTION_SAFETY_MAP = {
...
"my_action": SafetyLevel.SAFE,
}
```

Step 4: Add test

```
# tests/test_my_action.py
def test_handler_registered():
assert "my_action" in ACTION_HANDLERS
```

Testing

```
python -m pytest tests/ -v # all tests
python -m pytest tests/test_llm.py -v # specific
python -m pytest tests/ --cov=actions # coverage
```

Code Style

- Python 3.11+ syntax, type hints on all functions
- Async-first patterns (asyncio everywhere)
- Docstrings: one-liners, no novels
- 88-character line length

12. Day-by-Day Milestones

Two-week sprint to complete Phase 2 and stabilize for pre-launch. All three team members work in parallel.

Day 1 (Mon)

- **Aradhya (Mac):** Fix ElevenLabs TTS streaming on macOS, verify overlay transparency
- **Aryan (Win):** Wire DeepSeek production API key, test all task_types end-to-end
- **Maneesh (Win):** Scaffold email_actions.py — SMTP send + IMAP check, register handlers

Day 2 (Tue)

- **Aradhya:** Implement self-improving loop: log analysis module, prompt refinement engine
- **Aryan:** Optimize memory injection — reduce top_k, add conversation summarization
- **Maneesh:** Complete email actions + tests, create EMAIL_SETUP.md doc

Day 3 (Wed)

- **Aradhya:** Self-improving loop: wire into event_loop, test with real conversations
- **Aryan:** Action chaining MVP — pipe action output into next action's params
- **Maneesh:** Scaffold todo_actions.py — add/list/complete/delete with JSON storage

Day 4 (Thu)

- **Aradhya:** Reduce end-to-end latency: profile pipeline, optimize hot paths
- **Aryan:** Action chaining: integrate with planner, test multi-step workflows
- **Maneesh:** Complete todo actions + tests, scaffold weather_actions.py (OpenWeatherMap)

Day 5 (Fri)

- **Aradhya:** Hot-reload config: watch settings.yaml, reload without restart
- **Aryan:** Token budgeting: track DeepSeek API costs per session, add limits
- **Maneesh:** Complete weather actions + tests, scaffold timer_actions.py

Day 6 (Sat)

- **Aradhya:** Security audit: review all DANGEROUS actions, harden shell_actions blocklist
- **Aryan:** Improve wake word accuracy: tune energy threshold, test noisy environments
- **Maneesh:** Complete timer/reminder actions + tests, all Phase 2 actions done

Day 7 (Sun)

- **ALL:** Integration testing: full pipeline test with all new actions on both platforms

Day 8-9

- **Aradhya:** E2E test suite: automated boot test, action test, TTS test on macOS
- **Aryan:** E2E test suite: same on Windows, fix platform-specific issues
- **Maneesh:** Write all remaining docs: FEATURES_SUMMARY.md, LAUNCH_PRIORITY_LIST.md

Day 10-11

- **Aradhya:** Performance optimization: target sub-3s latency, profile and fix bottlenecks
- **Aryan:** Memory cleanup: add TTL to old vectors, compress knowledge graph
- **Maneesh:** Update PROJECT_GUIDE.md to reflect all Phase 2 features, generate final PDF

Day 12-14

- **ALL:** Bug fixes, polish, final testing on both platforms, tag v0.3.0 release

13. Work Division

Aradhya

Platform: macOS | **Focus:** Architecture, core loop, voice, UI, agents

Module	Files
Utils	config.py, logger.py, platform.py
Core	event_loop.py, state_manager.py, task_orchestrator.py, daemon.py
Voice	tts_router.py, elevenlabs_tts.py, stream_player.py
Actions	action_router.py, safety.py, all 6 action modules
UI	ipc_bridge.py, entire tauri-app/
Agents	planner.py, executor.py, agent_manager.py
Integration	main.py (boot sequence, module wiring)
Phase 2	Self-improving loop, latency optimization, security audit

Aryan

Platform: Windows | **Focus:** Audio, LLM, memory, browser, coding agent

Module	Files
Audio	listener.py, asr.py, groq_whisper_backend.py, wake_word.py, interrupt_handler.py
LLM	router.py, prompt_manager.py, deepseek_provider.py
Memory	memory_manager.py, short_term.py, long_term.py, vector_store.py, graph.py
Browser	browser_agent.py, login_handler.py, scraper.py
Agents	coding_agent.py, base_agent.py
Phase 2	Action chaining, token budgeting, wake word tuning

Maneesh

Platform: Windows | **Focus:** New actions, integrations, documentation

Module	Files
Phase 2 Actions	email_actions.py, todo_actions.py, weather_actions.py, timer_actions.py
Tests	test_email.py, test_todo.py, test_weather.py, test_timer.py
Documentation	All setup guides, FEATURES_SUMMARY.md, PROJECT_GUIDE.md
Phase 2	Complete all 4 action modules + docs + tests

14. Suggested Architecture Improvements

Recommendations based on codebase audit:

Streaming TTS (chunked playback)

Current: buffer all audio then play. Better: play chunks as they arrive from ElevenLabs for instant response feel. Requires refactoring _play_sync to accept a stream.

Multi-modal input (vision)

Add LLM vision support: screenshot -> send image to DeepSeek/GPT-4V -> understand what's on screen. Enables 'What app is open?' and 'Click the blue button'.

Plugin / Extension system

Define an Action plugin interface: drop a .py file into actions/plugins/, auto-discovered at boot. Enables community contributions without touching core code.

Conversation summarization

When short_term buffer exceeds 20 messages, summarize older messages via LLM and inject the summary instead. Reduces token usage while preserving context.

Rate limiting / token budget

Track DeepSeek API token usage per session. Alert user when approaching daily budget. Auto-switch to shorter prompts when budget is low.

Hot-reload configuration

Watch settings.yaml with watchdog. Reload voice_id, wake_words, log_level without restarting IRIS. Critical for iterating on settings.

Structured logging + telemetry

Replace text logs with structured JSON. Add timing spans per pipeline stage. Generate per-session performance reports.

Graceful wake word upgrade path

Current: ASR-based text matching (1.5-3s latency). Plan: wire openwakeword ONNX backend for instant (~100ms) wake word detection once models are downloaded.

15. Troubleshooting

Problem	Solution
Wake word not detecting	Check mic permissions. Speak clearly. Restart IRIS.
DeepSeek API error 401	Invalid API key. Check configs/keys.env.
Groq ASR timeout	Check internet. Verify GROQ_API_KEY. Rate limit: wait 30s.
TTS no audio	Check ElevenLabs key. Check speaker volume. Check pyaudio install.
Tauri overlay not showing	First compile = 5-10 min. Check port 7788 is free.
Module import error	Run pip install -r requirements.txt. Delete __pycache__/.
ChromaDB corrupt	Delete ~/.iris/memory/chroma/ and restart (recreates DB).
IRIS stays in INTERACTIVE	Pull latest code. _task_worker now returns to IDLE.
PyAudio install fails (Win)	Use: pipwin install pyaudio
Laptop heating up	Pull latest: IDLE buffer increased to 3s + energy gate.

16. Roadmap

Phase	Scope	Target	Status
1	Core loop, audio, LLM, 12 actions, memory, UI overlay, agents	Apr 2026	<font color="#22C55E">DONE
2	Email, todo, weather, timer, self-improving loop, DeepSeek routing	May-Jun 2026	<font color="#F59E0B">IN PROGR
3	Performance (<3s latency), security hardening, E2E testing	Jun-Jul 2026	<font color="#64748B">PLANNED
4	Settings UI, calendar, smart home, action chaining	Q3 2026	<font color="#64748B">PLANNED
5	Plugin ecosystem, multi-language, advanced vision	Q4 2026	<font color="#64748B">FUTURE</
6	Mobile companion, enterprise, Linux full support	2027	<font color="#64748B">FUTURE</

17. FAQs

Q: Is IRIS free?

A: Yes — MIT licensed, open source, free to use and modify.

Q: Does IRIS need internet?

A: Yes for current setup. DeepSeek (LLM), Groq (ASR), and ElevenLabs (TTS) are all cloud APIs. Local-only mode may return in a future phase.

Q: Does IRIS send my data to the cloud?

A: Only to APIs you explicitly configure (DeepSeek, ElevenLabs, Groq). All local processing stays on your PC. Memory is stored locally.

Q: Does IRIS work on macOS?

A: Yes. Aradhya develops and tests on macOS. Platform-specific code is isolated in utils/platform.py.

Q: Can I add custom actions?

A: Yes — follow the 4-step guide in Development Guide. One file + two line registrations.

Q: How do I uninstall?

A: Delete the Iris/ folder. Config and memory are in ~/.iris/ — delete manually if desired.

18. Credits & Changelog

Core Team

Name	Role	Platform
Aradhya	Architecture, core loop, voice, UI, actions, agents	macOS
Aryan	Audio, LLM routing, memory, browser, coding agent	Windows
Maneesh	New actions, integrations, documentation, testing	Windows

Technologies

- Python 3.11+ (runtime) & Tauri v2 / Rust (UI overlay)
- Groq Whisper API (ASR) & ElevenLabs (TTS)
- DeepSeek V3 Flash + Pro (LLM reasoning)
- ChromaDB + BGE-M3 (vector memory) & NetworkX (knowledge graph)
- Playwright (browser automation) & WebSocket IPC (Python <-> Tauri)

Changelog

v0.2.1 (May 2026)

- Switched to DeepSeek-only routing (removed Ollama/Qwen2.5 local fallback)
- Replaced local Whisper with Groq cloud ASR (whisper-large-v3-turbo)
- Fixed ElevenLabs TTS streaming error (httpx read() fix)
- Fixed state machine: ACTING now returns to IDLE (was stuck in INTERACTIVE)
- Added energy gate + 3s buffer to reduce CPU load
- Enabled macOS transparent overlay (macOSPrivateApi)

v0.1.0 (April 2026)

- Initial release: core event loop, state machine, 12 action handlers
- Audio pipeline: wake word, ASR, TTS, interrupt handling
- Agent system: planner, executor, coding agent, agent manager
- Memory: short-term buffer, ChromaDB vectors, NetworkX graph
- Tauri UI overlay with animated border states
- Windows + macOS support

IRIS — Making your PC responsive to voice. Your assistant, your way.